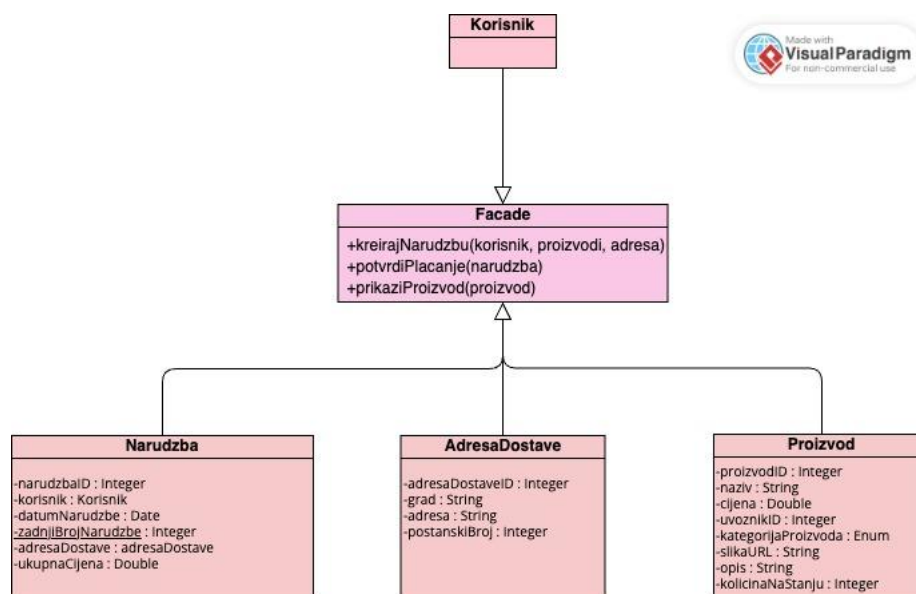


Strukturalni paterni

1. Facade Pattern

Primjena: pojednostavljenje interakcije korisnika sa složenim podsistemima.

- Klase poput Proizvod, Narudzba, Placanje, AdresaDostave, ValidacijaProizvoda imaju pojedinačne odgovornosti.
- Dodavanjem klase SistemFacade, korisnik (npr. Kupac ili Radnik) poziva jednostavne metode kao:
 - kreirajNarudzbu(...)
 - prikaziSveProizvode()
 - potvrdiPlacanje(...)

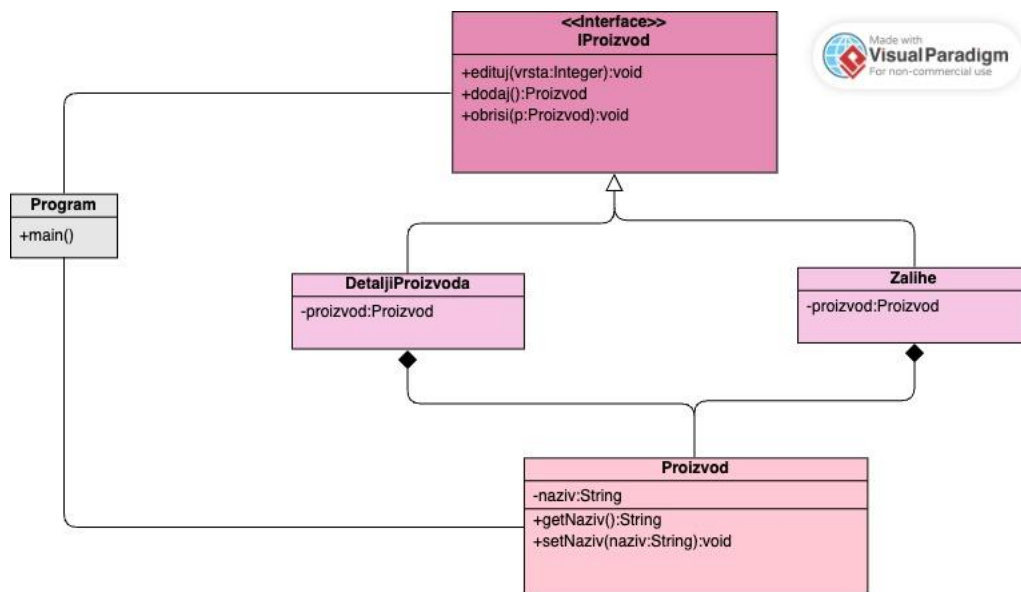


2. Decorator Pattern

Primjena: proširenje funkcionalnosti bez izmjene postojeće klase Proizvod.

- Radnik može dodavati nove attribute, upravljati zalihama, ažurirati opis itd.
- Uvodi se:
 - Interfejs IProizvod
 - Apstraktna klasa ProizvodDecorator
 - Konkretni dekoratori:
 - DetaljiProizvodaDecorator
 - ZaliheProizvodaDecorator

Omogućava radniku dinamično proširenje ponašanja proizvoda bez diranja glavne klase.



3. Adapter Pattern

Primjena: omogućava interakciju između klasa koje imaju nekompatibilne interfejsje.

- Klasa Proizvod možda ne sadrži sve potrebne metode za naprednu pretragu (npr. filtriranje po parametrima i cijeni).
- Uvodi se ProizvodAdapter koji implementira novi interfejs za naprednu pretragu proizvoda, koristeći podatke iz Proizvod.

Omogućava povezivanje različitih pretraga bez izmjene postojećih klasa.

4. Proxy Pattern

Primjena: kontrola pristupa i sigurnosti, posebno za osjetljive akcije.

- Radnik može imati posebne privilegije (npr. validacija proizvoda, pregled finansija).
- Uvodi se klasa RadnikProxy koja:
 - Provjerava da li radnik ima prava prije izvršavanja metoda (npr. odobriNarudzbu(), izmijeniCijenu()).

Štiti sistem od neautorizovanog pristupa osjetljivim funkcijama.

5. Bridge Pattern

Primjena: omogućava nezavisni razvoj apstrakcije i implementacije.

- Pregled proizvoda je moguć i za prijavljene i za neprijavljene korisnike.
- Napraviti apstrakciju PregledProizvoda, sa implementacijama:
 - GostPregled
 - KupacPregled

Korisno za razdvajanje ponašanja gosta i registrovanog korisnika.

6. Composite Pattern

Primjena: omogućava rad sa grupama objekata kao sa jednim objektom.

- Narudžba se sastoji od više ProizvodKorpa stavki.
- Uvodi se interfejs IArtikal koji sadrži metodu cijena().
- Narudžba poziva ukupnaCijena() iterativno ili rekurzivno kroz sve stavke.

Pogodno za buduće proširenje (npr. paketi, popusti).

